

ATMT Assignment 3

Giovanni Rocci (22-729-156) & Omkar Ingale (24-716-466)

Task 1: Retraining

We decided to implement Joint-BPE and train a model using the same tokenizer for source and target language. First, as suggested by Sennrich et al. (2016), we obtained the training data for the tokenizer by simply concatenating the content of the two raw text files (`train.cz` and `train.en`). We copied the `preprocess.py` script and created a new version: `preprocess_shared_voc.py`. The new preprocessing script accepts a single tokenizer model and, accordingly, just a single vocabulary size argument. It also instantiates a single tokenizer instance from the `shared_tokenizer.BPETokenizer` class. The function `merge_training_data()` handles the tokenizer training data concatenation, and gets called on the input to the training function of the BPE tokenizer. The `BPETokenizer` class in `shared_tokenizer.py` is very similar to the one in `tokenizer.py`, except for the presence of two different language attributes, one for the source and one for the target. The original class had only one language attribute because it was designed to build a tokenizer for a specific language.

Having applied the above modifications, we set up the retraining shell script (`retrain_sharedBPE.sh`). The shell script called `preprocess_shared_voc.py`, `train.py` and `translate.py`. The first script used the concatenated training data to train a joint-BPE tokenizer, which was then used to build binary datasets for training. We opted for a vocabulary size of 16,000, which is twice the size of the single-language tokenizers⁷. This decision was made in light of what was stated on slide 20 of Lecture 3, i.e., that usually for sets of subwords D holds that $\text{len}(D_{src}) = \text{len}(D_{trgt}) = 1/2 * \text{len}(D_{joint})$. Hyperparameters for the training and translation script calls were left almost identical to the ones from `assignment1.sh`, except for the fact that both source tokenizer

and target tokenizer arguments pointed to the same joint tokenizer model.

Figure 1 shows the validation perplexity and BLEU score for each training epoch. Last two BLEU score values (epoch 8 and 9) actually correspond to test set BLEU scores.

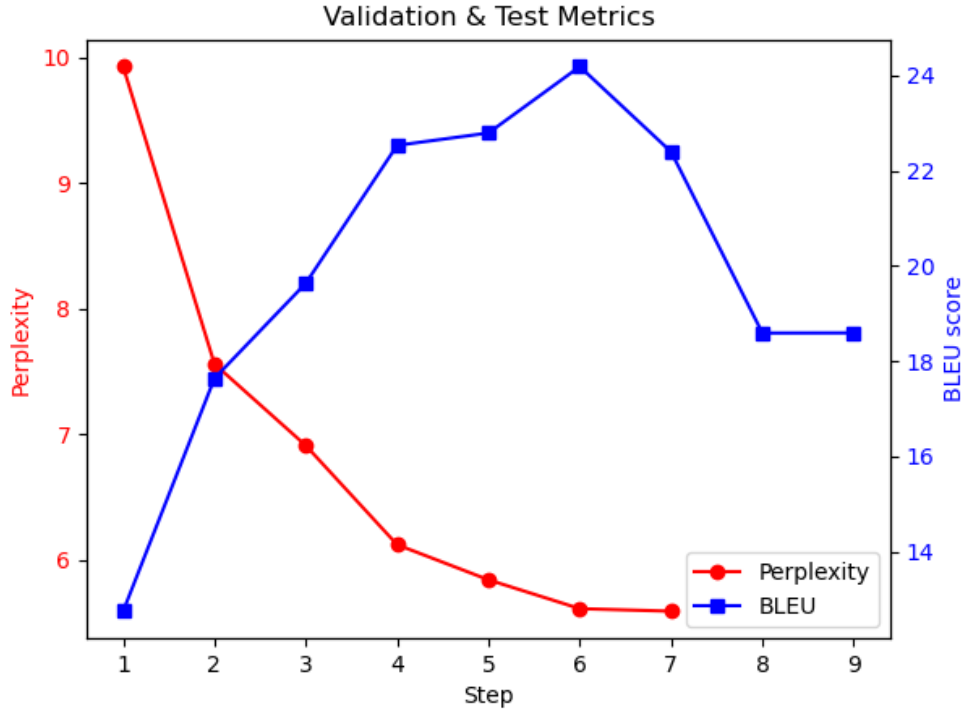


Figure 1: Validation perplexity and BLEU score for Joint-BPE model

The training time was not substantially different to the baseline; both models took around 18 hours to complete seven training epochs. This is not surprising, given that there were no architectural changes between the two versions. Achieved BLEU scores on validation and test set however showed interesting differences, as Figure 3 illustrates.

Despite achieving lower scores earlier on in training, and even not having the overall highest score, the Joint-BPE model obtains a higher BLEU score at test time compared to the baseline model. It is not a big difference (~ 2

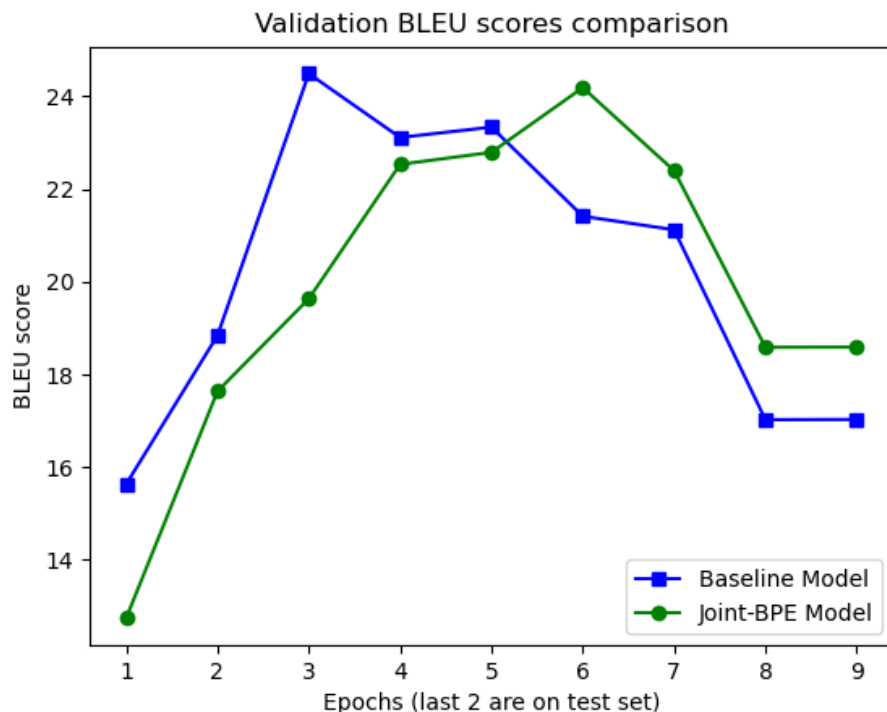


Figure 2: Validation and Test BLEU score for Joint-BPE model and baseline model (model from Assignment1)

points), but still indicates an improvement. This improvement is further confirmed by BLEU scores obtained when running `translation.py`. The Joint-BPE model achieves a BLEU score of 18.48, compared to 16.94 for the baseline model.

Analyzing the produced translations, differences are visible also to the human eye. Translations from the Joint-BPE model sound more fluent and are definitely less repetitive, as evidenced by the examples in Table 1

In the first example, the expression "are now turning around in the graves" sounds more plausible and fluent compared to "now turn to the graves". In the second example, the phenomenon is clearly more pronounced

Sent	Baseline model	Joint-BPE model
1	Your ancestors now turn to the graves.	and the ancestors are now turning around in the graves.
2	OGNY! But how? But how? How? How? How? How? How? How? How? How? How? How? How? How?	But how do you? But how? But how?

Table 1: Example sentences extracted from `output.txt` for both models.

in the baseline model’s translation, despite repetitions in both models (the Joint-BPE model translation was not free from repetitions either).

In conclusion, using the same joint tokenizer for the source and target sides seems beneficial for model performance. However, the different sizes of the tokenizers’ vocabulary are still worth investigating, as they represent another potential influential variable. It would be interesting to see if doubling the vocabulary size of the two monolingual tokenizers (or halving that of the joint tokenizer) would level the performance difference shown in figure 3.

Task 2: Refining

For the second task, we chose to refine our model from Assignment 1 and implement beam search. To ensure compatibility, we implemented a fallback mechanism. Whenever the `--beam_size` argument is passed to the `train.py` or `translate.py` script and it’s value is more than 1, we decode using beam search. Else, we fall back to the previous implementation of greedy search.

To test our implementation, we created 5 different scripts: `beamsearch3.sh`, `beamsearch5.sh`, `beamsearch7.sh`, `beamsearch10.sh`, and `greedysearch.sh` to test various beam sizes and compare them to the greedy search baseline. Each of these scripts were tested with `translate.py` on `test.cz` and reference on `test.en`. The results of the experiment are shown in table 2:

Decoding Strategy	BLEU Score	Time (s)
Greedy	19.49	47.587
Beam, Size = 3	29.58	35.327
Beam, Size = 5	32.73	37.021
Beam, Size = 7	34.03	42.923
Beam, Size = 10	28.87	54.290

Table 2: Comparison of BLEU scores and time (s) for inference for different decoding strategies

The following graphs 3, 4 demonstrate this information visually.

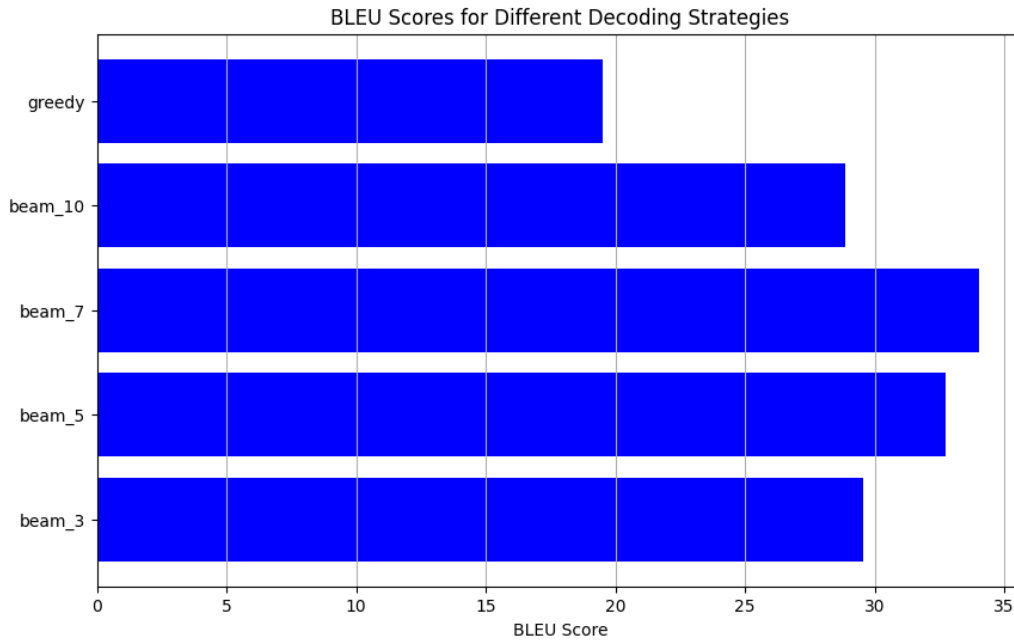


Figure 3: BLEU Scores for different decoding strategies

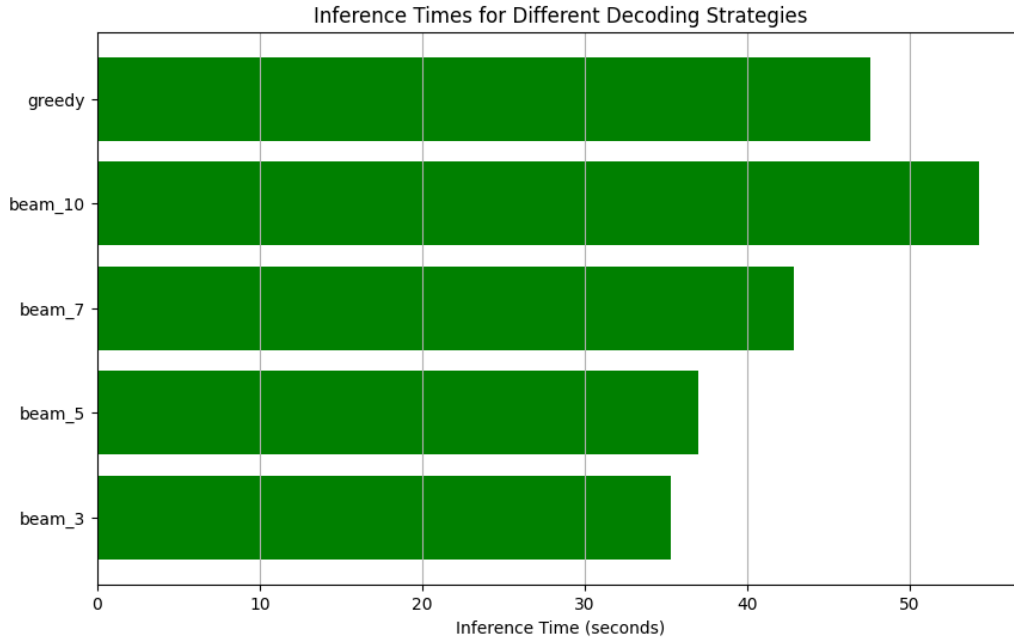


Figure 4: Time for inference with different decoding strategies

As can be seen, beam size 7 optimizes the BLEU score while having a small overhead in terms of time compared to beam sizes 3 and 5. The following tables 3, 4 demonstrate some examples of decoded sentences for each decoding strategy.

Decoding Strategy	Decoded Sentence
Greedy	If you are concerned about your health, please consult your doctor, you should always consult your doctor. If you are concerned about your health, please consult your health.
Beam, all sizes	If you are concerned about your health, please always consult your doctor.

Table 3: Decoded Examples 1

Decoding Strategy	Decoded Sentence
Greedy	patio, Ironheart, we had problems with him for telepath, on behalf of Ironheart, who we had problems with him for telepath, on behalf of Ironheart.
Beam, size = 3	person Ironheart, we had problems with him because of the telepath named Ironheart.
Beam, size = 5	: We had problems with him for the telepath named Ironheart.
Beam, size = 7	Ironheart had problems with him on behalf of Ironheart.
Beam, size = 10	.We had problems with him for the telepath named Ironheart.

Table 4: Decoded Examples 2

In conclusion, even though beam search outperforms simple greedy search, beam size plays an important role in how well the model is able to decode it’s predictions. A larger beam size does not necessarily lead to better BLEU scores (as seen by scores for size 7 and size 10). An optimal beam size needs to be calculated by balancing priorities between performance metrics and time spent on decoding.

GitHub repository

All the code used to produce the results presented in this report can be found at the following GitHub repository.

References

Sennrich, R., Haddow, B., and Birch, A. (2016). Neural machine translation of rare words with subword units.